FACULTAD DE INGENIERÍA

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

ARQUITECTURA DE SOFTWARE

Actividad: Historias de usuario y requerimientos

PRESENTADO POR:

- **Oyola Gutierrez Jerzeell Juniors**
- **Ccente Garcia Kevin**
- **Tovar Montoya Benjamin Joaquin**

Docente:

RAUL ENRIQUE FERNANDEZ BEJARANO

HUANCAYO - PERÚ

2026

1. Introduccion :

En la IEP La Cantuta Milenium radica la ausencia de un sistema automatizado para el control de asistencia de sus docentes, ya que actualmente, el proceso se realiza manualmente , lo que lo convierte en un método ineficiente, propenso a errores y con ello trae consecuencias como los errores en la manipulación del registro así generando una falta de precisión en la asistencia de los docentes y dificultando la efectividad en el control de horarios o las inasistencias injustificadas ,la consolidación de asistencia es mas laborioso para el encargado ya que debe recopilar y archivar las hojas físicas y consolidar la asistencia manualmente .

El control de asistencia de dicha institución es un problema significativo lo cual afecta al funcionamiento y a la calidad educativa en la institución generando así inconvenientes y preocupaciones en los estudiantes ya que la falta de los docentes afecta directamente al aprendizaje de los estudiantes ya que la educación requiere continuidad y consistencia para que los alumnos asimilen los conocimientos de manera efectiva.

2. Actores:

- **Docente:** Es el usuario que ingresará al sistema para registrar su propia asistencia (CUS3), consultar su horario (CUS9) y revisar su historial de asistencias (CUS10). También gestionará su propia cuenta (Iniciar Sesión, Cambiar Contraseña).
- **Secretaria:** Es la usuaria administrativa con roles operativos. Es responsable de gestionar el padrón de docentes (CUS7), administrar los horarios (CUS2) y registrar las justificaciones de inasistencia (CUS8).
- **Director:** Es el usuario de gestión o supervisión. Su función principal en el sistema es consultar y generar reportes de asistencia (CUS1) para la toma de decisiones.

- **Promotor :** Es el usuario de interés estratégico o el dueño de la institución. Su función principal en el sistema es la supervisión de alto nivel y la toma de decisiones financieras. Consulta y genera reportes de asistencia (CUS1) y estadísticas para evaluar el rendimiento del negocio y la optimización de recursos.

1. Objetivos del Negocio

Los objetivos originales se han adaptado para reflejar el uso de la nueva tecnología biométrica:

- **ON01 (Confiabilidad):** Optimizar la confiabilidad del registro de asistencia implementando **tecnología de reconocimiento facial**, erradicando el fraude,

contraseñas compartidas y las suplantaciones del proceso manual para mejorar la gestión académica institucional.

- **ON02 (Eficiencia):** Minimizar los tiempos operativos mediante un marcaje facial instantáneo sin contacto físico, y automatizar la generación de informes periódicos para dirección y administración.
- **ON03 (Reglas):** Estandarizar las políticas de horarios, procesando la validación del rostro en tiempo real para clasificar automáticamente y de forma uniforme el estado del docente como Presente, Tardanza o Inasistencia.
- **ON04 (Seguridad):** Asegurar la trazabilidad y auditoría de cada evento de marcaje garantizando que el rostro capturado coincida biométricamente con el padrón del docente, guardando los metadatos (rostro validado, fecha-hora, dispositivo, resultado).

1.3. Metodología de Desarrollo: Programación Extrema (XP)

Para el desarrollo del Sistema de Control de Asistencia Docente mediante Reconocimiento Facial se adoptó la metodología Extreme Programming (XP), ya que permite responder con rapidez a los cambios en los requisitos, favorece el desarrollo incremental y promueve la entrega continua de software funcional. Esta metodología es ideal para proyectos con equipos pequeños y donde la precisión funcional del sistema es crítica, como en el caso del motor biométrico de reconocimiento facial.

Principios XP aplicados al proyecto:

- **Juego de la Planificación:** Las funcionalidades se definieron como Historias de Usuario (HU) que describen el comportamiento esperado del sistema desde la perspectiva del docente y del administrador, priorizadas según su impacto en el control de asistencia.
- **Entregas Pequeñas:** El sistema se dividió en dos módulos entregables e independientes: registrar.py (enrolamiento facial) y verificar.py (marcaje diario), permitiendo validar cada uno por separado antes de integrarlos.
- **Diseño Simple:** Se evitó la complejidad innecesaria utilizando archivos CSV como persistencia de datos, carpetas del sistema operativo como base de datos de rostros, y parámetros configurables en constantes para adaptarse a distintas instituciones sin modificar el código.
- **Pruebas de Aceptación:** Cada Historia de Usuario incluye criterios de aceptación concretos y medibles, validados mediante pruebas reales con la cámara, verificando que el sistema reconozca correctamente al docente, rechace rostros no registrados y clasifique la asistencia según el horario configurado.

1.4. Historias de Usuario

HU-01: Marcaje de Asistencia Biométrico

Historia de Usuario	
Como	Docente de la institución.
Quiero	Registrar mi asistencia posicionándome frente a la cámara del sistema presionando la tecla V.
Para	Registrar mi entrada de forma rápida, segura y sin necesidad de contacto físico o firmas manuales.
MoSCoW	Must
Criterios de Aceptación	
CA1:	Al presionar V el sistema activa el análisis del rostro en el encuadre actual de la cámara.
CA2:	Si el rostro coincide con el padrón, muestra el nombre del docente, hora exacta y estado (A TIEMPO / TARDE / MUY TARDE).
CA3:	Si el rostro no es reconocido, el sistema indica “Docente no registrado” y permite reintentar.
CA4:	El sistema rechaza el intento si no detecta exactamente un rostro humano en el encuadre.

HU-02: Enrolamiento Facial del Docente

Historia de Usuario	
Como	Personal administrativo que ejecuta registrar.py.
Quiero	Capturar 5 fotografías del docente desde distintos ángulos y guardarlas en el sistema.
Para	Crear el patrón biométrico necesario para que el docente pueda marcar su asistencia posteriormente.
MoSCoW	Must
Criterios de Aceptación	

CA1:	El sistema valida que haya exactamente un rostro en el encuadre antes de permitir la captura.
CA2:	Las 5 fotos se guardan en una carpeta individual (fotos/nombre/) con nombres numerados.
CA3:	El sistema muestra un rectángulo verde al detectar rostro válido y un mensaje de progreso por cada foto capturada.

HU-03: Clasificación Automática del Estado de Asistencia

Historia de Usuario	
Como	Sistema de control de asistencia.
Quiero	Comparar la hora del marcaje facial con la hora oficial de entrada y clasificar el estado automáticamente.
Para	Uniformizar el criterio de puntualidad sin intervención manual, eliminando subjetividad en el registro.
MoSCoW	Must
Criterios de Aceptación	
CA1:	El estado es “A TIEMPO” si el marcaje ocurre en o antes de HORA_ENTRADA.
CA2:	El estado es “TARDE (TOLERADO)” si supera la hora pero está dentro de TOLERANCIA_MIN.
CA3:	El estado es “MUY TARDE” si excede la tolerancia configurada.

HU-04: Prevención de Registros Duplicados

Historia de Usuario	
Como	Sistema de control de asistencia.
Quiero	Impedir que un docente quede registrado más de una vez en el mismo día.
Para	Garantizar la integridad del historial de asistencias y la confiabilidad del archivo CSV.
MoSCoW	Must

Criterios de Aceptación	
CA1:	Al iniciar, el sistema carga los nombres ya registrados en la fecha actual.
CA2:	Si un docente reconocido ya tiene registro del día, el sistema muestra una advertencia y descarta el duplicado.
CA3:	El archivo asistencia.csv no es modificado en caso de intento duplicado.

HU-05: Generación de Registro de Asistencia en CSV

Historia de Usuario	
Como	Administrador o director de la institución.
Quiero	Contar con un archivo CSV actualizado automáticamente con el historial de marcajes del día.
Para	Revisar la asistencia de los docentes sin necesidad de sistemas adicionales o registros manuales.
MoSCoW	Should
Criterios de Aceptación	
CA1:	Cada marcaje exitoso agrega una fila al CSV con: Nombre, Fecha-Hora (formato 12h AM/PM) y Estado.
CA2:	El archivo se crea automáticamente si no existe al primer marcaje del día.
CA3:	El CSV puede abrirse directamente en Excel para análisis o impresión.

2. Requerimientos Funcionales (Enfoque Reconocimiento Facial)

A continuación se detallan los requerimientos funcionales del sistema implementado con Python, OpenCV y DeepFace:

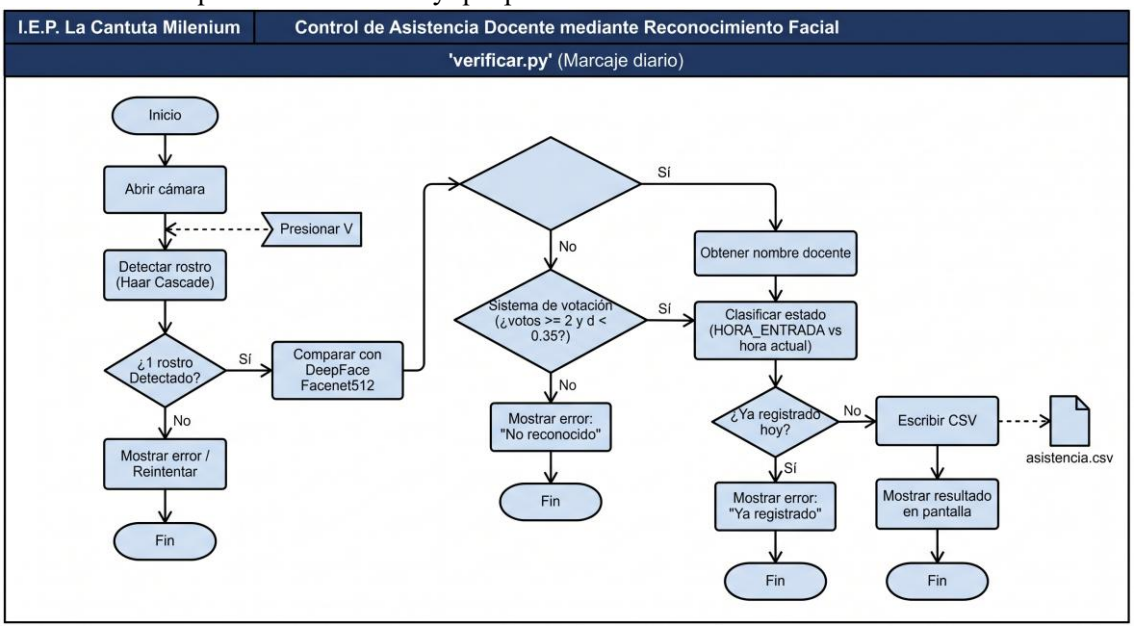
Código	Nombre del Requerimiento	Descripción
RF-001	**Reconocimiento facial con DeepFace**	El sistema captura el rostro en tiempo real con OpenCV y lo compara contra el padrón usando DeepFace (modelo Facenet512, métrica coseno). El docente es reconocido

		solo si al menos 2 de sus 5 fotos tienen distancia menor a 0.35.
RF-002	**Clasificación automática de asistencia**	Compara la hora del marcaje con HORA_ENTRADA y TOLERANCIA_MIN para clasificar en: “A TIEMPO”, “TARDE (TOLERADO)” o “MUY TARDE” automáticamente.
RF-003	**Registro en archivo CSV**	Cada marcaje exitoso escribe en asistencia.csv los campos: Nombre, Fecha-Hora (formato 12h AM/PM) y Estado. El archivo se crea automáticamente si no existe.
RF-004	**Enrolamiento facial múltiple**	El módulo registrar.py captura 5 fotos del docente validando un rostro por captura. Las imágenes se almacenan en fotos/nombre/nombre_N.jpg para su posterior uso en verificación.
RF-005	**Validación previa de rostro**	Antes del reconocimiento biométrico, Haar Cascade verifica que haya exactamente 1 rostro humano. Rechaza el intento con mensaje diferenciado si hay 0 o múltiples rostros.
RF-006	**Sistema de votación para identificación**	Compara el rostro contra las 5 fotos de cada docente. Identifica al candidato con mayor número de votos (coincidencias) y menor distancia promedio, evitando falsos positivos.
RF-007	**Retroalimentación visual en tiempo real**	La ventana de la cámara muestra un rectángulo de detención (verde/rojo), mensajes codificados por color y el resultado del último intento sobreimpreso en el video.
RF-008	**Estructura de carpetas por docente**	Organiza las imágenes en subcarpetas individuales (fotos/nombre/). El módulo verificar.py las recorre automáticamente con os.walk sin necesidad de configuración adicional.
RF-009	**Stack tecnológico Python**	Implementado en Python 3 con: OpenCV (captura y detección), DeepFace/Facenet512 (reconocimiento), datetime (cálculo de estados) y csv (persistencia). Compatible con equipos sin GPU.
RF-010	**Prevención de duplicados diarios**	Al iniciar, carga en memoria los docentes ya registrados hoy (registrados_hoy). Si se detecta un duplicado, muestra advertencia y descarta el registro sin modificar el CSV.
RF-011	**Parámetros configurables centralizados**	HORA_ENTRADA, TOLERANCIA_MIN, UMBRAL_DISTANCIA (0.35), VOTOS_NECESARIOS (2) y FOTOS_POR_DOCENTE (5) son constantes en la cabecera de cada módulo, fácilmente ajustables.

RF-012	**Módulos independientes**	registrar.py (enrolamiento, se ejecuta una vez por docente) y verificar.py (marcaje diario) son independientes, lo que permite actualizar el padrón sin interrumpir la operación.

Diagramas de Casos de Uso

Este diagrama ofrece una visión general de alto nivel de las interacciones entre los actores y el sistema. Define quién usa el sistema y qué pueden hacer con él.



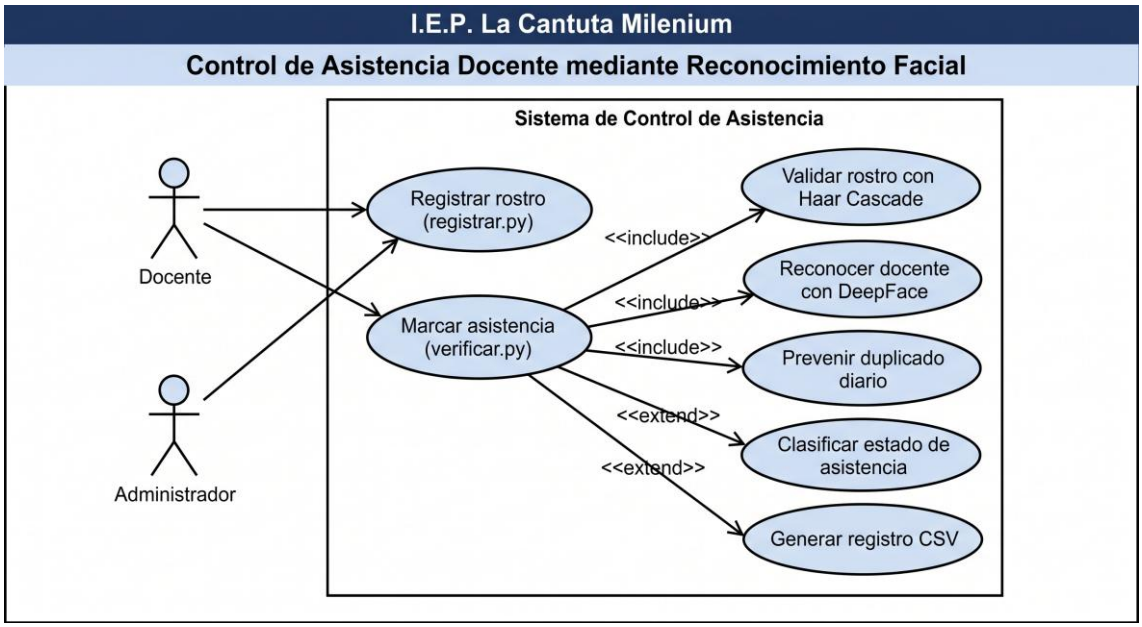
- **Actores:** Docente (inicia la acción) y Administrador (gestiona el registro).
- **Relaciones:** Se han incluido las relaciones UML estándar como <<include>> para los subprocesos obligatorios (validación y reconocimiento) y <<extend>> para las acciones condicionales (clasificación y generación de registro).

2. Diagramas de Flujo (Actividad)

Estos diagramas detallan la lógica procedimental de los dos módulos principales. He utilizado la notación UML de Diagrama de Actividad, que es la forma estándar de representar flujos de trabajo detallados .

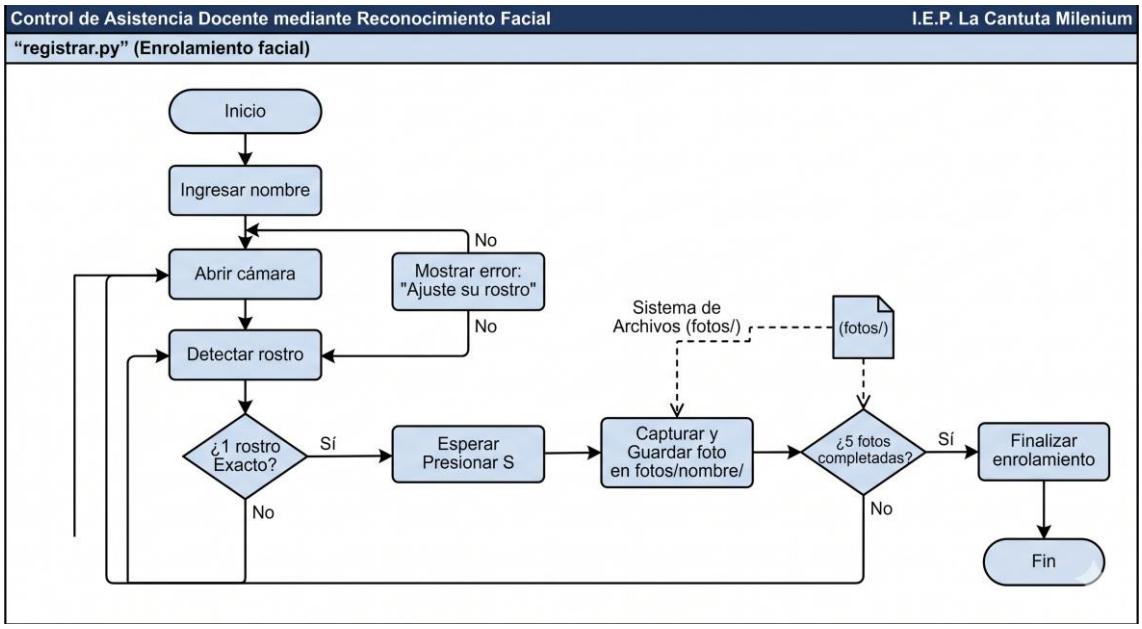
Flujo del proceso verificar.py (Marcaje Diario)

Este diagrama detalla toda la lógica compleja de la asistencia diaria, incluyendo la detección, la comparación con DeepFace, el sistema de votación y las clasificaciones de puntualidad.



Flujo del proceso registrar.py (Enrolamiento Facial)

Este diagrama muestra el flujo simple pero crucial de captura y validación de las 5 fotos necesarias para el enrolamiento de un docente.



3. Diagramas de Estructura (Componentes y Clases)

Estos diagramas muestran cómo está construido el sistema desde el punto de vista del software y de los datos.

Diagrama de Componentes

Muestra la arquitectura física del software, ilustrando cómo los scripts de Python dependen de bibliotecas externas (OpenCV, DeepFace) y de los archivos de datos (fotos y CSV). He incluido la estructura interna de DeepFace para mostrar el modelo y la métrica utilizados.

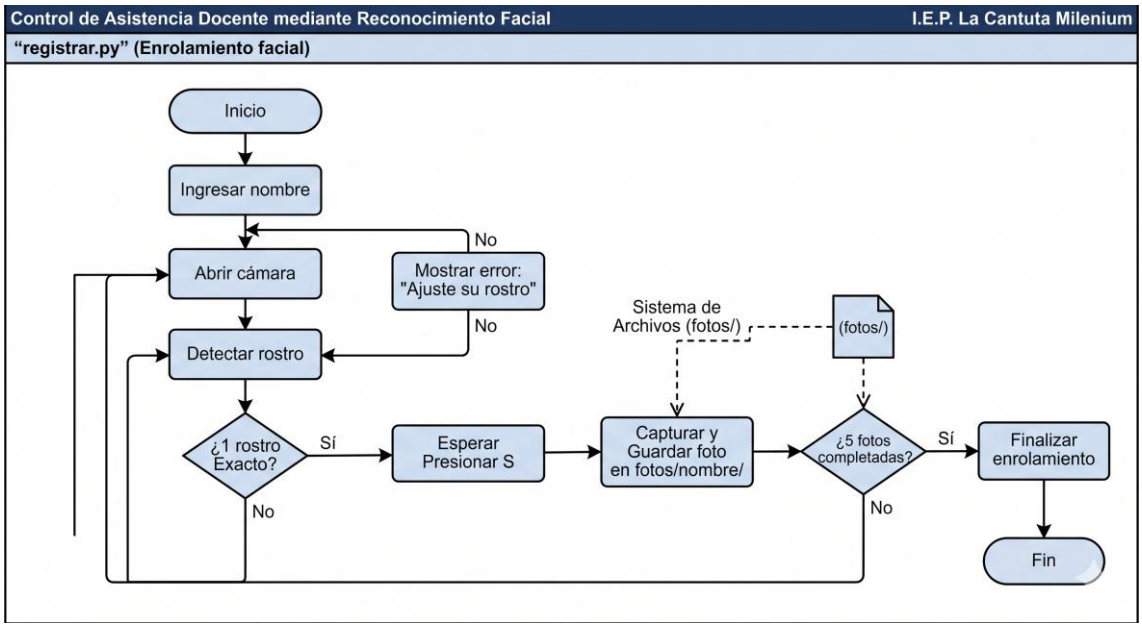
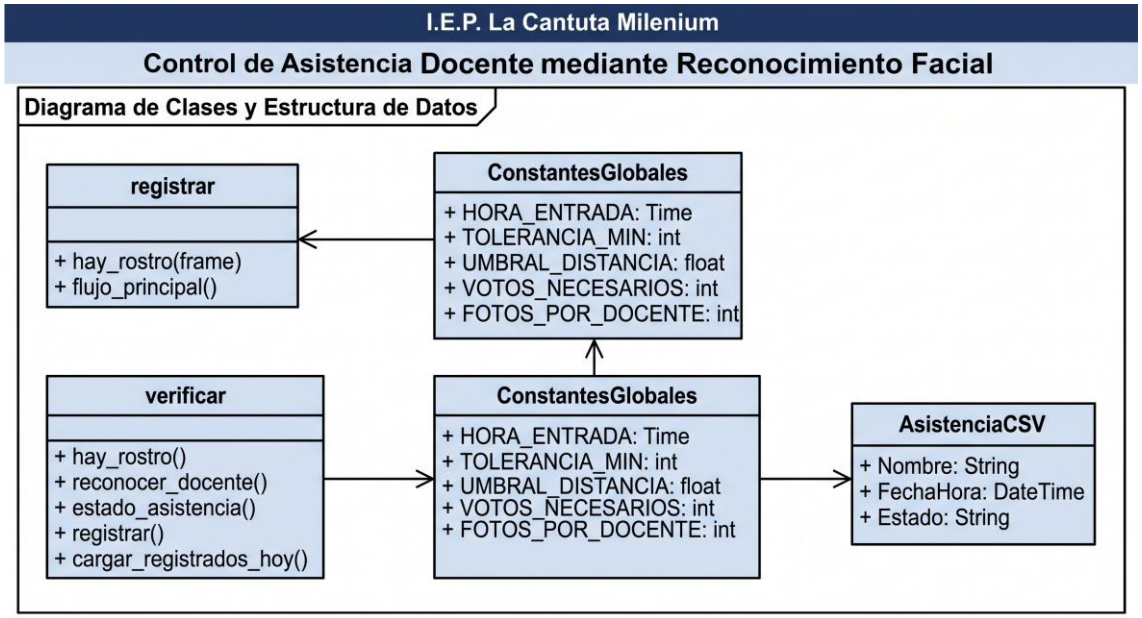


Diagrama de Clases / Estructura de Datos

Este diagrama detalla la estructura lógica del software. Muestra las funciones principales de cada script (representadas como métodos), la organización de las constantes globales y la estructura exacta de los campos del archivo asistencia.csv.